

Build a Redis Cluster Lifecycle Tool with Ansible

You will build a **CLI tool** that wraps **Ansible** to provision, operate, and perform a **rolling upgrade** of a Redis Cluster — with zero downtime and verified data integrity.

The tool manages a **6-node Redis Cluster** (3 masters + 3 replicas) running inside containers (Docker or Podman) that simulate real servers with SSH access. Everything is driven through your CLI — no manual SSH, no manual `redis-cli` commands during operations.

Infrastructure Setup

Before you write any automation, you need a local environment that simulates 6 servers.

What to Build

Create a compose file (`docker-compose.yml` or `podman-compose.yml`) that spins up **6 containers**:

- Base image: Ubuntu 22.04 (or similar)
- Each container runs an **SSH server** (not Redis directly — your Ansible playbooks install Redis)
- All containers on the same container network
- SSH key-based authentication from your host (control node) to all containers — **no passwords**

Your containers should be named something like:

```
redis-node-1  (10.10.0.11)
redis-node-2  (10.10.0.12)
redis-node-3  (10.10.0.13)
redis-node-4  (10.10.0.14)
redis-node-5  (10.10.0.15)
redis-node-6  (10.10.0.16)
```

Tip: Use a static subnet in your compose file so your Ansible inventory has fixed IPs.

Your host machine is the Ansible **control node**. The 6 containers are the managed nodes. This mirrors how Ansible works in production — SSH into remote servers and execute tasks.

The Tool — What to Build

Build a CLI tool called `redis-tool` (Bash, Python, Go, or any language). It must support the following commands. Each command orchestrates one or more Ansible playbooks under the hood.

Phase 1 — Provision a Redis Cluster

```
./redis-tool provision --version 7.0.15 --masters 3 --replicas-per-master 1
```

This command must:

1. **Install Redis 7.0.15** on all 6 nodes via Ansible
 - Build from source or install from packages — your choice, but it must be the exact version specified
2. **Configure Redis** on each node for cluster mode
 - `cluster-enabled yes`, `cluster-config-file`, `cluster-node-timeout`, appropriate `bind` and `port` settings
3. **Start Redis** on all nodes
4. **Form the cluster** — assign 3 nodes as masters (with hash slot ranges) and 3 as their replicas
5. **Print the final cluster topology** — which node is master/replica, slot ranges, and connectivity status

At the end of this command, you should have a fully operational Redis Cluster.

Phase 2 — Seed Data and Verify

```
./redis-tool data seed --keys 1000
./redis-tool data verify
```

data seed must:

1. Generate **1000 key-value pairs** with deterministic, reproducible values (e.g., `key:0001` → SHA256 of `key:0001`, or any scheme where the value can be independently recomputed)
2. Insert them into the cluster — keys will distribute across all masters via hash slots
3. Print a summary: total keys inserted, distribution across masters, any failures

data verify must:

1. Read back all 1000 keys from the cluster
 2. Recompute expected values and compare
 3. Print a result: `PASS - 1000/1000 keys verified` or `FAIL - X keys missing, Y values mismatched`
 4. This command will be run **before and after** the upgrade to prove data integrity
-

Phase 3 — Cluster Status

```
./redis-tool status
```

This command must print a clear, readable summary of the cluster:

- Each node's **IP, port, role** (master/replica), and **Redis version**
- For masters: **hash slot range** and **number of keys**
- For replicas: **which master they're replicating**
- Cluster state: `ok` or `fail`
- Memory usage per node

Example output:

```
Cluster State: ok
```

MASTERS

10.10.0.11:6379	[master]	v7.0.15	slots: 0-5460	keys: 332	mem: 2.1M
10.10.0.12:6379	[master]	v7.0.15	slots: 5461-10922	keys: 341	mem: 2.0M
10.10.0.13:6379	[master]	v7.0.15	slots: 10923-16383	keys: 327	mem: 1.9M

REPLICAS

10.10.0.14:6379	[replica]	v7.0.15	replicating: 10.10.0.11:6379	mem: 2.1M
10.10.0.15:6379	[replica]	v7.0.15	replicating: 10.10.0.12:6379	mem: 2.0M
10.10.0.16:6379	[replica]	v7.0.15	replicating: 10.10.0.13:6379	mem: 1.9M

Phase 4 — Rolling Upgrade (The Core Challenge)

```
./redis-tool upgrade --target-version 7.2.6 --strategy rolling
```

This is the most important part of the project. The upgrade must achieve **zero client-visible downtime**. The cluster must remain in an `ok` state throughout the entire process.

Required strategy:

1. Pre-flight checks

- Verify cluster is healthy (`cluster_state:ok`)

- Verify all nodes are reachable
- Verify the current version differs from the target version
- Run `data verify` to establish a pre-upgrade integrity baseline

2. Upgrade replicas first (one at a time)

- For each replica node:
 - Stop Redis on the node
 - Install the new Redis version (7.2.6)
 - Start Redis with the same configuration
 - Wait for the replica to rejoin the cluster and complete sync
 - Verify cluster state is `ok` before moving to the next node
- Print progress after each node: `[2/6] Upgraded replica 10.10.0.15 - cluster: ok`

3. Upgrade masters (one at a time, with failover)

- For each master node:
 - Trigger `CLUSTER FAILOVER` on its replica — the replica becomes the new master (promotes the upgraded replica)
 - Wait for failover to complete
 - The old master is now a replica — stop Redis on it
 - Install the new Redis version (7.2.6)
 - Start Redis, wait for it to rejoin as a replica
 - Verify cluster state is `ok` before moving to the next master
- Print progress after each node

4. Post-upgrade verification

- Run `data verify` — all 1000 keys must still be present and correct
- Run `status` — all nodes must show the new version
- Print `UPGRADE COMPLETE` — all nodes on v7.2.6, data integrity verified

If any step fails, the tool must:

- Stop the upgrade immediately (do not continue to the next node)
- Print which node failed and at which step
- Leave the cluster in whatever state it's in (do not attempt automatic rollback unless you implement the bonus)

Phase 5 — Full Verification

```
./redis-tool verify --full
```

A comprehensive post-upgrade health check:

1. **Data integrity** — same as `data verify`
2. **Version consistency** — all nodes report the same Redis version
3. **Topology health** — all hash slots covered (16384 total), every master has at least one replica
4. **Cluster state** — `cluster_state:ok`
5. **Replication lag** — all replicas have `master_link_status:up`

Print a pass/fail summary for each check.

Stretch Goals (Optional)

These are not required but will be noted positively:

S1 — Scale Out

```
./redis-tool scale --add-nodes 2
```

Add a new master + replica pair to the cluster. Rebalance hash slots across all masters.

S2 — Scale In

```
./redis-tool scale --remove-node <node-id>
```

Migrate all slots off a master, remove it and its replica from the cluster.

S3 — Rollback

If the upgrade fails mid-way, provide a command to roll back upgraded nodes to the previous version:

```
./redis-tool rollback --target-version 7.0.15
```

S4 — Idempotency

Running `provision` on an already-provisioned cluster should be a no-op (or update config without data loss). Running `upgrade` when all nodes are already at the target version should print a message and exit cleanly.

S5 — Structured Logging

Write a detailed operation log (JSON or structured text) to `logs/` for every command. Include timestamps, node names, actions taken, and outcomes.

What to Submit

```
submission/
├── redis-tool                ← CLI entrypoint (bash/python/go/etc.)
├── ansible/
│   ├── ansible.cfg
│   ├── inventory/
│   │   └── hosts.ini        ← or hosts.yml
│   ├── playbooks/
│   │   ├── provision.yml
│   │   ├── upgrade.yml
│   │   ├── status.yml
│   │   └── ...
│   └── roles/
│       └── redis/
│           ├── tasks/
│           ├── handlers/
│           ├── templates/
│           └── defaults/
├── infra/
│   ├── Containerfile        ← Ubuntu + SSH base image (Dockerfile also accepted)
│   └── compose.yml          ← 6-node cluster infra (Docker Compose or Podman Compose)
├── README.md
└── output/
    ├── provision_output.txt  ← paste of actual terminal output
    ├── data_seed_output.txt
    ├── status_output.txt
    ├── upgrade_output.txt
    └── verify_output.txt
```

README.md Must Cover

- How to bring up the container infrastructure (works with Docker or Podman)
 - How to run each `redis-tool` command
 - The rolling upgrade strategy you implemented and why
 - Any assumptions or trade-offs you made
 - Known limitations
-

Rules

- **Write your own Ansible playbooks and roles** — do not use Ansible Galaxy roles or pre-built collections for Redis
- You may use Ansible built-in modules (`command` , `shell` , `template` , `copy` , `apt` , `service` , `wait_for` , etc.)
- **No managed Redis services** — no ElasticCache, no Redis Cloud, no Kubernetes operators
- The CLI tool can be written in any language — it orchestrates Ansible, it does not replace it
- Data operations (seed/verify) can use `redis-cli` on the nodes (via Ansible) or direct TCP commands — your choice
- Your tool must support both **Docker Engine** and **Podman** as the container runtime — use whichever is available on the host (Podman is preferred since it's fully open source with no enterprise licensing concerns)
- The submission must run on **macOS or Linux** with no assumptions about pre-installed tools (see prerequisite check below)

Prerequisite Check

Your tool must include a **prerequisite check** as the very first thing it does on any command. Before executing any logic, `redis-tool` must verify that the required dependencies are present on the host machine:

1. **Container runtime** — Docker Engine or Podman (either is acceptable). If both are present, prefer Podman. Podman is fully open source (Apache 2.0) with no enterprise licensing concerns — recommend it in install instructions.
2. **Ansible** — `ansible-playbook` must be available and at version 2.14+

Behavior:

- If all prerequisites are found, print their versions and continue:

```
✓ Podman 4.9.3 found
✓ Ansible 2.16.4 found
Proceeding...
```

- If any prerequisite is **missing**, print a clear message telling the user exactly what to install and how, then **exit with a non-zero code**. Do not attempt to auto-install anything without user consent.

```
x Container runtime not found (Docker or Podman)
  Install Podman:  https://podman.io/docs/installation
  Install Docker:  https://docs.docker.com/engine/install/

x Ansible not found
  Install:  pip install ansible  (or use your OS package manager)

Please install the missing dependencies and retry.
```

- **Optional (stretch):** Offer to install missing dependencies automatically if the user passes a `--auto-install` flag. If you implement this, it must ask for confirmation before making system changes.

Note on Podman vs Docker: Podman is a drop-in replacement for Docker for container management. `podman-compose` handles compose files. The key difference for this project: Podman runs rootless by default, which may require adjusting how you start the SSH server inside containers. Your tool should detect which runtime is available and use the appropriate commands (`docker` / `docker compose` or `podman` / `podman-compose`).